

6.3 Function and Class Documentation

In function and class documentation, you view functions or classes as a unit and describe them. In both cases, you explain the purpose of the respective structure.

You use function and class documentation to give developers an overview of the structure. This documentation provides a compact description of all the important aspects you need to know about the structure to use it effectively. The requirements for function and class documentation differ in detail.

The most important information you should generally provide includes the following:

- **Brief description**

Give a one-sentence explanation that captures the purpose of the structure and mentions what you can do with it and why it exists.

- **Detailed description**

Add details in addition to the aspects of the brief description.

- **Examples**

You can specify how a function or class is to be used and add a short code example.

- **Known errors or to-dos**

If there are known restrictions, errors or things that still need to be done, the documentation block is a good place to document this.

For a class, you can also specify dependencies or information about the main author.

In the case of function documentation, in addition to the preceding information, you also describe the parameters, the return value, and any exceptions the function can throw.

There are standards for class and function documentation in the various programming languages. In the case of TypeScript and JavaScript, this is JavaScript Documentation (JSDoc).

6.3.1 Benefits and Drawbacks of Function and Class Documentation

Compared to inline documentation, function and class documentation provides better context as it's applied to an entire code block. This also makes it clearer and provides a good overview. However, good function and class documentation is time consuming and carries the risk of quickly becoming outdated. This is exactly where AI tools can help.

You'll find this type of documentation much more frequently than inline documentation. Especially in areas where the source code is made available to other teams or projects in the form of libraries, there is no way around this type of documentation. Developer manuals are also often generated automatically from the comment blocks. To keep the effort to a minimum, often only the external interfaces are documented. Many developers don't provide any detailed documentation for internal or private classes, methods, and functions.

6.3.2 Sample Function and Class Documentation

The Person TypeScript class serves as a concrete example of function and class documentation. Here, you can see not only how you can document a class but also what this can look like for a function, as it makes hardly any difference whether you document a function or a method of a class.

The source code of the class reads as follows:

```
class Person {
  private name: string;
  private age: number;

  constructor(name: string, age: number) {
    this.name = name;
    this.age = age;
  }
}
```

```

    greet(): string {
        return `Hello, my name is ${this.name} and
            I am ${this.age} years old.`;
    }
}

```

With this source code, you can ask the AI tool of your choice, such as ChatGPT, to add the documentation, which you can do via the following prompt:

Prompt: Please add the missing class and method documentation to the following class:

[code of the class]

Here's the Person class with added JSDoc comments for class and method documentation:

```

/**
 * The `Person` class represents a person with a name
 * and age.
 */
class Person {
    private name: string;
    private age: number;

    /**
     * Creates an instance of the `Person` class.
     *
     * @param {string} name - The name of the person.
     * @param {number} age - The age of the person.
     */
    constructor(name: string, age: number) {
        this.name = name;
        this.age = age;
    }

    /**
     * Returns a greeting message including the person's
     * name and

```

```

    * age.
    *
    * @returns {string} A string containing the
greeting
    * message.
    */
greet(): string {
    return `Hello, my name is ${this.name} and
        I am ${this.age} years old.`;
}
}

```

Summary of the documentation:

- Class Level: Describes the purpose of the Person class.
- Constructor: Documents the parameters required to create a Person object.
- Method: Describes the greet() method and its return value.

Even this simple example shows that you can save yourself a lot of work by letting the AI tool create the class and function documentation for you. To ensure that prompts like this work smoothly, you should make sure your code is cleanly structured. The naming of parameters also helps AI generate the documentation. If you also use a type-safe language such as TypeScript, you have the option of specifying the types of the parameters and the return value. The more information you give the AI tool, the better the result will be.

As always when working with AI, you should at least skim the generated source code and make sure that the comment blocks match the functions. However, AI tools such as ChatGPT can also derive very well from the code and generate the documentation, even if the code isn't really cleanly written, as the following example of a binary search shows:

```

function s(a, b) {
    let l = 0, r = arr.length - 1;
    while (l <= r) {
        const m = Math.floor((l + r) / 2);
        if (arr[m] === target) return m;
    }
}

```

```
        if (arr[m] < target) l = m + 1;
        else r = m - 1;
    }
    return -1;
}
```

You can use the following prompt to generate a comment block for the function:

Prompt: Could you please write a documentation for the following function:

[code of the function]

Certainly! Here's a detailed documentation for the s function:

```
/**
 * Performs a binary search on a sorted array to find
 * the index of a target element.
 *
 * ...more comments here...
```

[explanation: more details]

Here, too, the AI tool has done a great job and created a very detailed commentary block. It contains both a description of the parameters and the return value as well as examples of use and a detailed description of the function. In addition, the tool has added a note that the input array must be sorted; otherwise, the behavior is undefined.